

1. Objective

The objectives of this experiment are:

- To understand the concept of the FIRST function in context-free grammars.
- To determine the set of terminals that can appear first in strings derived from a grammar symbol.
- To understand the role of FIRST sets in top-down and predictive parsing.
- To calculate FIRST sets recursively for terminals and non-terminals.
- To implement a basic FIRST function using C++.
- To handle epsilon (ϵ) productions while computing FIRST sets.

2. Introduction

In compiler design, syntax analysis determines whether a sequence of tokens follows the grammar of a programming language. Top-down parsing techniques need information about which terminal symbols may appear first in strings derived from a grammar symbol. This information is represented by the FIRST function.

For a grammar symbol X , $\text{FIRST}(X)$ contains the terminals that can occur as the first symbol of any string derived from X . If X can derive the empty string, epsilon (ϵ) is also included in $\text{FIRST}(X)$.

The FIRST function is especially important in predictive parsing and LL(1) parser construction because it helps a parser choose the appropriate production based on the next input token.

3. Theoretical Background

3.1 FIRST Function

For a grammar symbol X , $\text{FIRST}(X)$ is the set of terminals that can begin strings derived from X . The following rules are commonly used:

- If X is a terminal, then $\text{FIRST}(X) = \{X\}$.
- If X is a non-terminal and $X \rightarrow \epsilon$, then ϵ is included in $\text{FIRST}(X)$.
- If $X \rightarrow Y_1 Y_2 \dots Y_n$, add $\text{FIRST}(Y_1)$ except ϵ to $\text{FIRST}(X)$.
- If Y_1 can derive ϵ , then add $\text{FIRST}(Y_2)$ except ϵ , and continue in the same way.

- If every symbol $Y_1...Y_n$ can derive ϵ , then ϵ is included in $FIRST(X)$.

3.2 Why FIRST is Required

A predictive parser uses FIRST sets to determine which production should be selected for a non-terminal. If the next input token belongs to the FIRST set of a particular production, the parser can use that production without trying unrelated alternatives.

FIRST sets are also used together with FOLLOW sets when constructing LL(1) parsing tables, particularly when epsilon productions are present.

4. FIRST Function Rules

$FIRST(a) = \{ a \}$ where a is a terminal

If $A \rightarrow \epsilon$, then $\epsilon \in FIRST(A)$

If $A \rightarrow BC$, then $FIRST(B) - \{\epsilon\} \subseteq FIRST(A)$

If $\epsilon \in FIRST(B)$, then $FIRST(C)$ may also contribute to $FIRST(A)$

The process continues from left to right through the right-hand side until a symbol is found that cannot derive ϵ .

5. Algorithm

Step 1: Start with an empty result set for $FIRST(X)$.

Step 2: Check whether X is a terminal. If it is a terminal, add X to the result and return.

Step 3: Find all productions whose left-hand side is X .

Step 4: For each production, examine the symbols on the right-hand side from left to right.

Step 5: If the current symbol is a terminal, add it to $FIRST(X)$ and stop processing that production.

Step 6: If the current symbol is a non-terminal, recursively calculate its FIRST set and add its non-epsilon symbols to $FIRST(X)$.

Step 7: If the recursive FIRST set contains ϵ , continue to the next symbol in the production.

Step 8: If all symbols in the right-hand side can derive ϵ , add ϵ to $\text{FIRST}(X)$.

Step 9: Remove duplicate symbols from the final result.

Step 10: Display $\text{FIRST}(X)$.

6. Worked Example

Consider the following grammar:

$$A \rightarrow BC$$

$$B \rightarrow b$$

$$C \rightarrow c$$

We want to calculate $\text{FIRST}(A)$.

Step 1: $A \rightarrow BC$, so the first symbol on the right-hand side is B.

Step 2: Calculate $\text{FIRST}(B)$. Since $B \rightarrow b$, b is a terminal.

$$\text{FIRST}(B) = \{ b \}$$

Step 3: B cannot derive ϵ , so we do not continue to C for this production.

Step 4: Therefore, b is the only terminal that can begin a string derived from A.

$$\text{FIRST}(A) = \{ b \}$$

This matches the output shown in the supplied laboratory example.

7. Worked Example with Epsilon

Consider:

$$A \rightarrow BC$$

$$B \rightarrow b \mid \epsilon$$

$$C \rightarrow c$$

Since B can produce b or ϵ , $\text{FIRST}(B)$ contains both b and ϵ .

$$\text{FIRST}(B) = \{ b, \epsilon \}$$

Because B can disappear through ϵ , C must also be considered. Since $C \rightarrow c$, c is added to FIRST(A).

$$\text{FIRST}(A) = \{ b, c \}$$

This demonstrates why the FIRST function must continue to the next symbol when the FIRST set of the current non-terminal contains ϵ .

8. C++ Implementation

The following C++ program follows the recursive idea shown in the supplied screenshots. Productions are stored as strings, uppercase letters represent non-terminals, lowercase letters represent terminals, and \$ is used internally to represent epsilon.

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;
int numProductions;
string productionSet[20];
// Add a symbol only if it is not already present
void addToResultSet(string &result, char value)
{
    if (result.find(value) == string::npos)
        result += value;
}
// FIRST function
void FIRST(string &result, char c)
{
    // If c is a terminal
    if (!isupper(c))
    {
```

```

addToResultSet(result, c);
return;
}
// Check every production of the grammar
for (int i = 0; i < numProductions; i++)
{
// Production belongs to the non-terminal c
if (productionSet[i][0] == c)
{
// X -> epsilon
if (productionSet[i][2] == '$')
{
addToResultSet(result, '$');
continue;
}
bool allNullable = true;
// Examine RHS from left to right
for (int j = 2; productionSet[i][j] != '\0'; j++)
{
string subResult;
FIRST(subResult, productionSet[i][j]);
bool hasEpsilon = false;
// Add FIRST(symbol) except epsilon
for (char ch : subResult)
{
if (ch == '$')
hasEpsilon = true;
else
addToResultSet(result, ch);
}
}
}
}

```

```

// Stop if this symbol cannot derive epsilon
if (!hasEpsilon)
{
    allNullable = false;
    break;
}

// If every RHS symbol can derive epsilon
if (allNullable)
    addToResultSet(result, '$');
}
}

int main()
{
    char choice;
    char symbol;
    string result;
    cout << "Enter number of productions: ";
    cin >> numProductions;
    for (int i = 0; i < numProductions; i++)
    {
        cout << "Enter production " << i + 1 << ": ";
        cin >> productionSet[i];
    }
    do
    {
        cout << "\nFind FIRST of: ";
        cin >> symbol;
        result.clear();
    }
    while (choice != 'q');
}

```

```

FIRST(result, symbol);
cout << "FIRST(" << symbol << ") = { ";
for (char ch : result)
{
    if (ch == '$')
        cout << "epsilon";
    else
        cout << ch;
    cout << " ";
}
cout << "}" << endl;
cout << "\nContinue (y/n)? ";
cin >> choice;
} while (choice == 'y' || choice == 'Y');
return 0;
}

```

9. Program Explanation

Production storage: The productionSet array stores the grammar productions entered by the user.

addToResultSet(): This function prevents duplicate terminals from being added to the FIRST set.

FIRST(): This is the main recursive function. It handles terminals directly and recursively evaluates non-terminals.

Terminal checking: isupper() is used to distinguish non-terminals from terminals in the simplified representation.

Epsilon handling: The program uses \$ internally to represent ϵ . When a production derives epsilon, \$ is added to the intermediate FIRST set.

Recursive processing: For a production such as $A \rightarrow BC$, $FIRST(B)$ is calculated first. If B can derive ϵ , $FIRST(C)$ is considered as well.

Output: The final unique symbols are printed as $FIRST(X)$.

10. Sample Input

```
Enter number of productions: 3
```

```
Enter production 1: A=BC
```

```
Enter production 2: B=b
```

```
Enter production 3: C=c
```

```
Find FIRST of: A
```

11. Sample Output

```
FIRST(A) = { b }
```

```
Continue (y/n)? n
```

12. Output Interpretation

For the input grammar:

$$A \rightarrow BC$$

$$B \rightarrow b$$

$$C \rightarrow c$$

The program first evaluates $FIRST(B)$. Since B directly produces the terminal b, $FIRST(B) = \{b\}$. Because B cannot derive ϵ , the parser does not need to examine C for the FIRST set of A. Therefore:

$$FIRST(A) = \{ b \}$$

13. Example Demonstrating Epsilon

Suppose the grammar is:

$$A \rightarrow BC$$

$$B \rightarrow b \mid \epsilon$$

$$C \rightarrow c$$

The computation proceeds as follows:

$$\text{FIRST}(B) = \{ b, \epsilon \}$$

$$\text{FIRST}(C) = \{ c \}$$

$$\text{FIRST}(A) = \{ b, c \}$$

The important point is that ϵ in $\text{FIRST}(B)$ allows the computation to continue to C . Thus c also becomes a possible first terminal of strings derived from A .

14. Advantages of FIRST Function

Helps determine which terminals can begin strings derived from a grammar symbol.

Provides essential information for predictive parsing.

Helps construct LL(1) parsing tables.

Reduces uncertainty when selecting productions in top-down parsing.

Supports systematic handling of epsilon productions.

15. Limitations of the Basic Program

The program uses a simplified representation where uppercase characters are treated as non-terminals and lowercase characters as terminals.

The implementation is intended for educational demonstration rather than a complete parser generator.

It does not construct a complete LL(1) parsing table.

It does not calculate FOLLOW sets.

For complex grammars, additional safeguards may be required to avoid recursive cycles.

16. Discussion

In this experiment, the FIRST function of a context-free grammar was studied and implemented using C++. The FIRST set identifies the terminals that may occur at the beginning of strings derived from a grammar symbol. This information is essential for top-down and predictive parsing.

The program uses recursion to calculate FIRST sets. When a terminal is encountered, it is directly added to the result. When a non-terminal is encountered, its FIRST set is calculated recursively. If epsilon is present in that FIRST set, the next symbol in the production is also examined.

For the example $A \rightarrow BC$, $B \rightarrow b$, and $C \rightarrow c$, the program correctly determines $\text{FIRST}(A) = \{b\}$. An additional epsilon example demonstrates that if B can derive ϵ , then $\text{FIRST}(C)$ must also contribute to $\text{FIRST}(A)$.

17. Conclusion

In this laboratory experiment, the FIRST function of a context-free grammar was studied and implemented in C++. The experiment demonstrated how FIRST sets are calculated for terminals and non-terminals and how recursive evaluation is used when a production contains multiple symbols.

The experiment also showed the importance of epsilon in FIRST-set computation. If a symbol can derive ϵ , the next symbol in the production may also contribute to the FIRST set. Therefore, the FIRST function is a fundamental component of predictive parsing and LL(1) parser construction.

18. References

- V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd Edition, Pearson.
- Compiler Design course lecture slides and laboratory materials: FIRST Function.
- Compiler Design Laboratory course instructions and class notes.